

Secure System Engineering Principles

Software developers are expected to adhere to Normain's coding standards throughout the development cycle, including standards for quality, commenting, and security. This document contains principles for security and privacy. Each principle description is followed by a short bullet list of examples intended to further clarify, and should not be used as a checklist.

Secure-by-Design Principles

1. Reduce Attack Surface

The attack surface of a system is the sum of all the different points where an unauthorized user could try to enter data or extract data. By disabling or removing unnecessary code, services, and interfaces, we minimize the potential entry points for attacks. This means reducing the complexity of the system and only including features that are absolutely essential for its intended function. Keeping the system lean and mean not only improves security but also often leads to better performance.

- Disable unused APIs/modules after setup.
- Only expose necessary ports/protocols.
- Close other ports (e.g., FTP, Telnet).

2. Apply Secure Defaults

Secure defaults mean configuring systems, permissions, and behaviors to be secure right out of the box. This ensures that users do not have to take extra steps to make the system secure. It sets a baseline of security that is active from the moment the system is deployed, reducing the risk of misconfigurations or accidental vulnerabilities. Pre-setting secure options reduces the burden on users and lessens the likelihood of security oversights.

- Default to strong password policies.
- Require single sign-on (SSO) for administrators and enable multi-factor authentication at the identity provider.
- Set default file storage to private.
- Enable data encryption by default.

3. Enforce Least Privilege

Least privilege dictates that users or processes should only have access to the minimum resources and permissions required to perform their intended functions. This limits the

potential damage if an account or process is compromised. By strictly controlling access, we reduce the risk of lateral movement by attackers and minimize the scope of damage from any single breach. Granular control is crucial for minimizing risk.

- Use role-based access control (RBAC).
- Assign permissions based on defined roles (e.g., User, Admin, Credit Manager).

4. Implement Defense-in-Depth

Defense-in-depth involves using multiple layers of security controls to protect assets. If one layer fails, another layer is there to catch the threat. This strategy increases the overall resilience of the system by providing redundant security measures. It acknowledges that no single security measure is foolproof and aims to create a layered shield of protection.

- Apply network and web application firewalls.
- Implement intrusion detection systems and endpoint security.
- Encrypt data at storage, in transit, and potentially in memory.
- Regularly scan for vulnerabilities and conduct penetration testing.

5. Ensure Systems Fail Securely

When a system experiences an error or exception, it should fail in a way that does not expose sensitive data or compromise security. Proper error handling and exception management are essential to prevent information leaks. Fail-safe defaults and secure error messages are crucial to maintain security during unexpected events.

- Display generic error messages to users.
- Log detailed error information internally for debugging.
- Invalidate and clear session data upon critical errors.

6. Treat All External Services as Untrusted

Any external service or input should be treated as potentially malicious. Validating all data and behaviors from external sources is crucial. This principle helps defend against supply chain attacks and data injection vulnerabilities. Always scrutinize external interactions and avoid making assumptions about the trustworthiness of outside data or services.

- Rigorously validate all incoming data for format and type.
- Use secure API keys and validate all responses from third-party APIs.
- Implement independent security checks for external services.

7. Separate Duties

Separating duties ensures that no single individual can complete a critical task without oversight. This reduces the risk of insider threats and collusion. By dividing responsibilities among multiple people or roles, you minimize the potential for abuse or fraud. This principle creates a system of checks and balances.

- Distinguish responsibilities across development, deployment/release, and security review.
- Prevent any single role from completing critical tasks without oversight.

8. Avoid Relying on Obscurity

Security should not depend on keeping implementation details secret. True security relies on robust design and implementation. Relying on obscurity is weak and easily defeated. Security through obscurity provides a false sense of security and can crumble when details are discovered.

- Focus on secure coding practices, strong authentication, and access control.
- Use standard encryption protocols rather than custom methods.
- Ensure transparent and auditable access control mechanisms.

9. Design Systems for Clarity and Simplicity

Systems should be easy to understand, maintain, and audit. Simplicity reduces the chance of errors and makes it easier to identify security flaws. Complexity often leads to oversights and vulnerabilities. The clearer the system is, the easier it is to spot and fix issues.

- Use well-documented, standard components for infrastructure.
- Avoid overly complex configurations.
- Write clear, well-commented code and descriptive variable names.
- Create easy-to-understand system architecture diagrams.

10. Correct Security Flaws Promptly

When a security vulnerability is discovered, it should be fixed as quickly and completely as possible. Having established remediation processes helps ensure that flaws are addressed efficiently. Rapid response to security issues minimizes the window of opportunity for attackers to exploit them. Proactive patching and vulnerability management are crucial.

- Establish a system for reporting and tracking vulnerabilities.
- Assign clear responsibility for investigating and patching flaws promptly.
- Automate vulnerability scanning and patch deployment where possible.

Privacy-by-Design Principles

1. Anticipate and Prevent Privacy Risks

This principle emphasizes proactively identifying and mitigating privacy risks early in the development lifecycle. It involves conducting privacy impact assessments (PIAs) and threat modeling to foresee potential issues before they become embedded in the system. By building in privacy protections from the start, we reduce the likelihood of privacy breaches and the need for costly retrofits later.

- Conduct thorough privacy impact assessments.
- Design systems with data minimization in mind.
- Consider hashing or removing identifiable information from data.

2. Default to Privacy-Protective Settings

This principle requires systems to have the most privacy-protective settings enabled by default, without requiring any action from the user. Users should not need to opt into privacy; privacy should be the standard. This approach ensures that users who may not be technically savvy or may overlook privacy settings are still protected. It also helps to establish a culture of privacy from the outset.

- Do not collect or store user activity logs by default.
- Anonymize or pseudonymize user data before use.
- Ensure personal data is not stored beyond processing time.

3. Integrate Privacy Safeguards Directly

Privacy safeguards should be embedded directly into the system architecture and data workflows, not added as an afterthought. This involves weaving privacy considerations into every aspect of system design and operation. Privacy should be a fundamental part of the system's DNA, ensuring consistent and comprehensive protection.

- Use data encryption at all stages.
- Implement robust access controls and logging.
- Incorporate data anonymization techniques into data processing.

4. Support Full Functionality while Protecting Privacy

Privacy-by-design is not about sacrificing functionality for privacy. The goal is to enable full functionality while ensuring strong privacy protections. This requires innovative design

thinking to find solutions that address both aspects. The aim is to create a win-win situation where users get both useful services and robust privacy.

- Minimize the personal data shared with third-party AI providers; aggregate or redact where feasible.
- Prefer aggregated or anonymized inputs when sending data to external AI services.

5. Secure Personal Data Across its Full Lifecycle

This principle requires ensuring the security of personal data from the moment it is collected through its eventual deletion. It includes secure storage, transmission, processing, and disposal of data. Data should be protected at every stage of its journey within the system.

- Use secure methods for data collection (e.g., HTTPS).
- Encrypt data at rest and use secure key management.
- Audit and update data retention policies; delete data securely when no longer needed.

6. Enable Transparency

Transparency involves making data practices understandable and accessible to users. Users should be informed about what data is collected, how it is used, and who has access to it. Clear and concise privacy policies and easily accessible information are crucial. Openness about data practices builds trust and empowers users to make informed decisions.

- Provide a clear and concise privacy policy.
- Provide users a means to access and manage their data on request.
- Explain how data is used for processing.

7. Design Interactions and Settings to Prioritize User Control

This principle focuses on giving users control over their privacy preferences. Users should be able to easily manage their privacy settings and decide how their data is collected, used, and shared. User-friendly interfaces and granular controls are essential. User empowerment is a key aspect of privacy-by-design.

- Provide detailed privacy settings for data control.
- Allow users to opt out of specific data processing activities.
- Offer tools for users to access, correct, or delete their data.

Document History

Version	Date	Description	Written by	Approved by
1.0.0	2025-06-04	Initial version	Gustav Lidén	Not recorded
1.0.1	2026-06-10	Annual review; examples corrected to reflect actual product (RBAC roles, third-party LLM usage, MFA posture, privacy claims).	Kalle Hansson	Kalle Hansson